DATA AND INFORMATION MANAGEMENT

ISYS2120

Revision Guide

Concepts, worked methods and exam preparation

Weeks 1-13 | Full English edition

Understand the facts. Design the schema. Derive the query. Prove the constraints.

Explanations, comparison tables, worked SQL, EER notation, closure calculations and a compact reference.

Use with the separate Interactive Question Bank: 100 Canvas questions plus 50 written practice tasks.

Semester 2, 2026 | Personal study edition

Prepared from course PDFs and visible completed Canvas quiz reviews. Explanations and new exercises are independently written study support.

Contents

Click a heading or use the PDF bookmark sidebar. Page numbers match the PDF reader.

How to use this guide 3

1. Foundations: data, DBMS and the relational model 4

2. Reading and designing EER models 6

3. Relational mapping and SQL constraints 9

4. SQL: build the result one logical step at a time 11

5. Advanced SQL: subqueries, sets and "every" 14

6. Relational algebra: derive the table you need 16

7. Functional dependencies, keys and normalisation 18

8. Security, privacy and integrity mechanisms 20

9. Database-backed applications and their security 22

10. Storage, indexing and query processing 24

11. Transactions, ACID and concurrency 26

12. Analytics and data governance 28

13. Integrated worked methods 30

14. Exam preparation and answer checks 32

15. Compact reference and glossary 34

16. Source index and verification notes 36

17. Practical SQL workshop | Ed Lessons 38

How to use this guide

This guide follows the structure of the INFO3315 revision material: explain the problem behind a concept, state the rule precisely, compare nearby ideas, work through an example, and finish with checks. It is written entirely in English so that the terminology matches ISYS2120 questions.

Three passes. Learn Chapters 1-12, work through Chapters 13-14, then attempt the separate Interactive Question Bank before opening answers. Its 150 tasks comprise 100 visible Canvas questions, 18 revision-paper subquestions, 24 skill drills and an eight-question practice exam. Repeated Canvas occurrences retain their own option order and submission evidence.

References. [L2A pp.41-43] means physical PDF pages counted from 1; source codes appear in Chapter 16. Worked examples and practice are independently written. Canvas labels record published answers, while technical corrections are marked separately.

Coverage. The local sources span Weeks 1-13, including the 2025-named transaction PDF used by the current Canvas module, available tutorials, the revision paper and Week 8 ZIP source. Chapter 16 records filenames and limits. Unseen quiz variants and a live university application run are not claimed.

Explicit exclusions. L2A p.44 excludes general min..max notation: use the unit's arrows and line thickness. L12 p.39 excludes GROUPING SETS; p.42 requires window concepts, not detailed syntax. The full SQL datatype catalogue is not a memorisation target.

The course in one connected story



A database design must preserve the facts that users need. SQL and relational algebra express requests about those facts. Constraints and access control protect validity and permitted use. Application code must preserve transaction boundaries and treat input as data. Physical organisation changes performance while leaving the logical answer unchanged. Analytics and governance extend these responsibilities across an organisation.

Study block	Main question	Practise until you can
Foundations	What is represented, and at which level?	Distinguish schema, instance, metadata, relation and relationship
Conceptual and logical design	What exists, and how are facts connected?	Draw unit-specific EER notation and map it to keys, FKs and tables
SQL and algebra	Which rows and attributes answer the request?	Derive joins, groups, subqueries and set expressions
Design theory	Which dependencies cause redundancy?	Compute closures, prove keys and justify decompositions
Security and applications	Who may do what, through which code?	Combine views and privileges; trace input to SQL execution
Systems and governance	How is data used efficiently, reliably and responsibly?	Explain access paths, ACID, OLAP and decision rights

1. Foundations: data, DBMS and the relational model

1.1 Why separate data from application code?

A **database** is an organised collection of persistent facts. A **DBMS** is the software that stores, retrieves, modifies and manages that collection. PostgreSQL is a DBMS platform; SQL is a language used to communicate with relational systems. A database can exist without a DBMS, such as a carefully organised set of files, but then the applications must implement much of the management themselves.

Consider a university where enrolment, timetabling and fee applications all use student information. If every application keeps its own copy of a student's address, a change can leave contradictory versions. Shared data reduces this duplication, but creates coordination, security and availability requirements. A DBMS centralises capabilities such as declared constraints, access checks, transaction recovery and query processing. This reduces repeated application work; it does not make systems free, automatically correct or immune to failure. [L1B pp.4-13, 40-49]

Data management is wider than storage. Someone must decide which facts to collect, who owns them, which uses are justified, how long to retain them, how to maintain their quality, and how to recover after failure. A technically fast query over the wrong or unlawfully reused facts is not good data management.

1.2 Schema, instance and metadata

The **schema** describes the structure and constraints: table names, columns, types, primary keys and foreign keys. An **instance** is the current content at a particular time. Inserting an enrolment changes the instance; adding a column changes the schema. A diagram of a schema does not show every current record.

Metadata is information that describes the data: the meaning and datatype of salary, its source, its update history or the rules controlling its use. The salary of one particular employee is an ordinary data value. Metadata can itself be stored in tables, such as the system catalogue. "Data about data" describes a role, not a special storage format. [L1B pp.18-21, 36]

Term	Example	Common confusion
Schema	Student(sid, name), PK(sid)	Not the current list of students
Instance	Rows (1, 'Mia'), (2, 'Noah')	Not the permanent design
Metadata	sid is an integer identifier	Not the sid value of a particular student
Database	Shared university data	Not the DBMS executable
DBMS	PostgreSQL server software	Not the SQL language

1.3 Relations, tuples, attributes and domains

In the mathematical relational model, a **relation** is a set of tuples over named attributes with appropriate domains. A tuple is one record of related values; an attribute is one named component; a domain is the permitted kind or range of values. Attribute order and tuple order are not essential to the mathematical meaning. A relation has no duplicate tuples because it is a set.

SQL approximates this model with tables. SQL can retain duplicate rows, use NULL and return an unordered result unless ORDER BY is specified. Do not assume that an ordinary SELECT eliminates duplicates. Use DISTINCT when the question asks for distinct values and the query can otherwise repeat them. [L4A pp.11-16; L2B pp.11, 31-34]

A **relation** in a relational database means a table-like mathematical structure. A **relationship** in an ER model means an association among entities. A relationship can become a separate relation, but a many-to-one relationship may instead be represented by an FK column.

1.4 Abstraction, independence and responsibilities

An **external view** presents information useful to a particular group. The **logical schema** defines the overall relational organisation. The **physical schema** concerns storage, indexes and access structures. Physical data independence allows storage choices to change without rewriting every query. Logical independence concerns insulating external uses from changes to logical structure, often through views; it is harder to guarantee for every change.

End users perform organisational tasks. Application programmers implement software and database interactions. The DBA manages access, backup, configuration, integrity and performance for the organisation. Vendor staff develop, support and improve the DBMS and its tools. Writing the organisation's specialised reporting application is normally an application programmer's task, rather than the definition of a DBMS vendor role. [L1B pp.25-33]

Explain why changing an index usually changes neither the schema seen by a user nor the intended query answer.

The index changes an access path at the physical level. The SQL query still requests the same logical result. An application that depends on incidental output order without ORDER BY has made an invalid assumption; that assumption can become visible when the plan changes.

2. Reading and designing EER models

2.1 What belongs in a conceptual model?

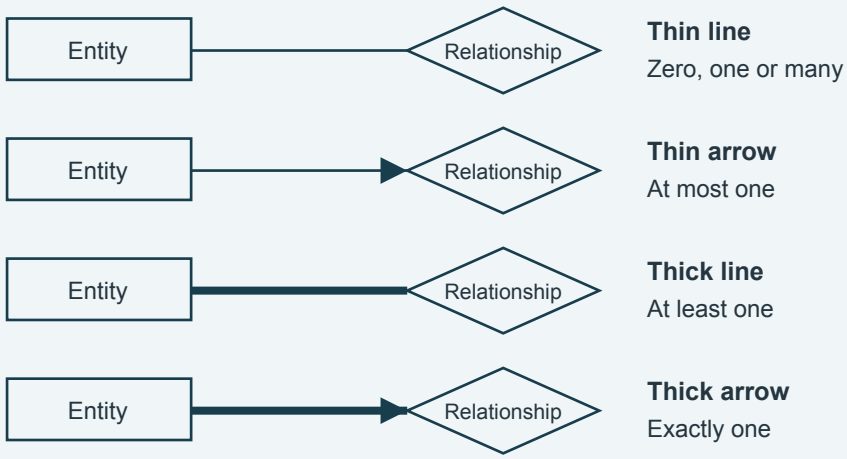
A conceptual model describes relevant domain facts before choosing a storage implementation. An **entity type** groups distinguishable things, such as Student, Workshop or Airplane. An **entity instance** is one particular student or workshop. An **attribute** describes an entity or relationship. A **relationship type** represents an association whose instances connect particular entities.

Use a rectangle for an entity type, an ellipse for an attribute and a diamond for a relationship type. Underline the identifier attributes. A **composite attribute**, such as name with given and family components, is drawn as a parent attribute connected to its component ellipses. A **multivalued attribute**, such as several contact numbers, uses a double ellipse. Composite does not mean multivalued: these are independent properties. [L2A pp.24-33]

The unit explicitly separates conceptual relationships from implementation foreign keys. If an apparent attribute is simply the identifier of another entity, draw a relationship to that entity rather than copying its identifier into the conceptual entity. The foreign-key column is introduced during relational mapping. A composite identifier, an attribute on a relationship and a multivalued entity attribute are all legitimate conceptual constructs. [L3B p.23; L5 p.7]

2.2 Read each side separately

For each entity type participating in a relationship, ask two questions: **what is the minimum number of relationship instances for one entity, and what is the maximum?** Read constraints from the relevant entity toward the diamond. In ISYS2120, arrowheads are at the diamond end.



For "Each employee works in exactly one department," draw a thick arrow from Employee to WorksIn. If a department may have zero or many employees, draw a thin plain line on the Department side. Do not put the arrow at Department just because it is called the "one side" in casual language. The arrow constrains the participation of the entity from which that arrow starts. [L2A pp.40-43]

At most one allows zero. **At least one** allows several. **Exactly one** combines both constraints. A statement that an entity "must participate" does not imply that it participates twice or more. This is the distinction behind the recurring Game-Competitor quiz question.

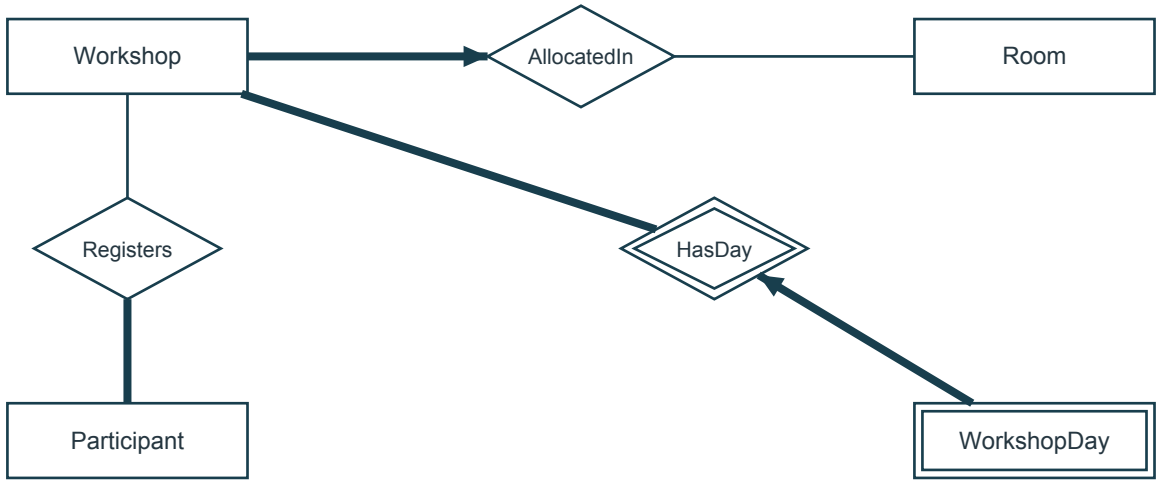
2.3 Requirements to a defensible model

Nouns suggest candidate entities or attributes; verbs suggest relationships. These are clues rather than mechanical rules. A noun becomes an entity when individual instances matter and have information or relationships to track. Words such as "must," "may," singular and plural provide clues about participation and cardinality. A verb connecting two nouns identifies an association, but alone does not specify its cardinality. [L5 pp.3-7]

Worked example: conference workshops. Each workshop uses one room for all its days. A workshop can meet on several dates, with a chairperson recorded separately on each date. Participants must register for one or more workshops; rooms are identified by floor and room number.

Represent Workshop with an assumed unique workshopID, Room with composite identifier (floor, roomNo), and Participant with an explicitly stated identifier such as a unique email. Use a thick arrow from Workshop to AllocatedIn, with Room on the other side. Registration is many-to-many with total participation for Participant. Represent WorkshopDay as a weak entity owned by Workshop, using dayDate as discriminator and chairperson as an attribute. This keeps chairperson dependent on a workshop-date combination rather than on the workshop alone. State any identifier assumption the text did not guarantee. [Adapted explanation of L13 pp.22-27]

High-level EER: attributes and identifiers are specified in the accompanying text.



Arrowheads face the relationship diamond. Thick edges require participation.

Do not invent a different room for each day when the text explicitly says the room remains the same. Do not make chairperson a multivalued attribute of Workshop if the pairing of chairperson and date would be lost. A model is adequate when the required facts and processes can be represented without ambiguity.

2.4 Weak entities, roles and history

A **weak entity** cannot be uniquely identified using only its own attributes. Its identity includes the owner's key plus a **discriminator**, which is unique only within that owner. Use a double rectangle, a double diamond for the identifying relationship, a dashed underline for the discriminator, and total many-to-one participation in the owner relationship. Payment number 3 can occur for several loans, but the pair (loanNumber, paymentNo) identifies one payment. [L2A pp.46-47]

A weak entity is not simply an entity with few attributes, or an entity whose key happens to be composite. Its identity depends on an owner. A recursive relationship uses the same entity type in different roles, such as manager and subordinate; name those roles so that later foreign keys have clear meanings.

History changes identity requirements. A Student-UoS relationship identified only by student and unit cannot represent two separate enrolments in the same unit unless another distinguishing item is modelled. If repeated

events matter, introduce an offering, event or attempt entity, or an appropriate additional discriminator. Do not add arbitrary history when the problem describes only the current state.

2.5 ISA and aggregation

A subclass is a subset of superclass instances and inherits the superclass attributes and relationships. CPU inherits Product's barcode and name, then adds cores, cache and speed. It does not inherit RAM-only attributes merely because RAM shares a superclass.

Disjoint versus overlapping concerns whether one superclass entity can belong to several subclasses. **Total versus partial covering** concerns whether every superclass entity must belong to some subclass. They are independent choices. The course defaults are overlapping and partial, but writing them explicitly avoids ambiguity. Use an ISA triangle; a thick line from superclass to triangle indicates total covering. [L2A pp.49-52]

Aggregation treats a relationship instance as something that can participate in another relationship. If a manager supervises a particular employee-job-branch assignment, connect the manager to that assignment rather than separately implying management of every possible combination. Reifying an association as an entity is another modelling approach when it has substantial attributes or identity. [L2A pp.53-55; L5 p.7]

A description says each room has a fixed capacity and each workshop-day has a chair. Where should those attributes go?

Capacity belongs to Room because it describes the room independently of workshops. Chair belongs to the workshop-day combination because it can vary by date within a workshop. Check attribute meaning before drawing lines.

3. Relational mapping and SQL constraints

3.1 Map information and constraints together

A logical schema is not complete just because every noun has a table. Specify primary keys, foreign keys, nullability and other stated constraints. A mapping must permit all valid domain states while excluding invalid states to the extent supported by the chosen declarations. [L3B pp.4-24]

Conceptual construct	Relational representation	Crucial constraint
Strong entity	One table with its simple attributes	Entity identifier becomes PK
Composite attribute	Store its relevant components	Do not duplicate an independent parent value unnecessarily
Multivalued attribute	Separate table: owner key plus value	Composite PK; owner FK
Weak entity	Owner key, discriminator and other attributes	PK(owner key, discriminator); owner FK
Many-to-many association	Join table with participant keys and relationship attributes	PK normally combines participant keys
Many-to-one association	FK in many-side table, or separate association table	FK; NOT NULL for total many-side participation
One-to-one association	FK with UNIQUE in a suitable table, or association table	FK uniqueness; nullability reflects participation
ISA	Superclass table and one table for each subclass	Subclass PK also FK to superclass

3.2 Many-to-many versus many-to-one

For Student(sid, name), Unit(ucode, title) and a many-to-many Enrolled relationship with grade, use Enrolled(sid, ucode, grade), PK(sid, ucode), and two FKs. The copied identifiers are not ordinary descriptive attributes of the conceptual relationship; they implement the participants' identities.

For Employee working in at most one Department, either use Employee(empID, name, deptID, since) or WorksIn(empID, deptID, since) with PK(empID). A separate association table whose PK is (empID, deptID) would allow the same employee in several departments, unless an additional UNIQUE(empID) constraint is added. When the FK is stored in Employee, total participation is represented by deptID NOT NULL. [L3B pp.11-15]

A foreign key from Employee to Department does **not** force every Department to have an Employee. Total participation on the referenced side often requires an additional mechanism, such as a suitable trigger or application-level transaction check. Identify this limitation rather than claiming the FK enforces it.

3.3 Weak entities, multivalued attributes and ISA

A room identified within a building maps to Room(buildingID, roomNo, capacity), PK(buildingID, roomNo), FK(buildingID) to Building. Room number alone is not globally unique. A composite owner key must be copied in full.

Several phone numbers for a person map to PersonPhone(personID, phone), PK(personID, phone), FK(personID) to Person. One row stores one number. This supports any number of values without inventing columns phone1, phone2 and phone3. A requirement that every person have at least one phone is not enforced by that FK alone.

For an ISA hierarchy, the standard mapping keeps Person(id, commonAttributes), Employee(id, salary) and Customer(id, loyaltyNo), with each subclass id both PK and FK to Person. This supports overlapping and partial hierarchies. The PK/FK pattern alone does not enforce disjointness or total covering. Alternative mappings need the relevant assumptions: subclass-only tables require total covering; a single discriminator table suits disjoint subclasses and introduces nullable subtype columns. [L3B pp.6-10, 18-21]

3.4 Write DDL that means what you intend

```
CREATE TABLE Department (
    deptID INTEGER PRIMARY KEY,
    deptName VARCHAR(80) NOT NULL
);
CREATE TABLE Employee (
    empID INTEGER PRIMARY KEY,
    empName VARCHAR(80) NOT NULL,
    deptID INTEGER NOT NULL REFERENCES Department(deptID),
    salary NUMERIC(10,2) CHECK (salary >= 0)
);
CREATE TABLE Assignment (
    empID INTEGER REFERENCES Employee(empID),
    projectID INTEGER,
    hoursWorked INTEGER NOT NULL CHECK (hoursWorked >= 0),
    PRIMARY KEY (empID, projectID)
);
```

Here Assignment's PK makes both empID and projectID non-NULL and makes the pair unique. It does not make empID alone unique. projectID has no FK in this short example because a Project table has not been declared; add that FK when Project exists. Do not label an undeclared relationship as enforced.

A **PRIMARY KEY** combines uniqueness and non-nullability. A **UNIQUE** constraint can express an alternative identifier, but does not by itself require values to be present. Use NOT NULL as well when modelling an alternative mandatory key. A **CHECK** condition such as salary >= 0 rejects false results; SQL NULL can produce unknown, so add NOT NULL if the value must always be supplied. [L3A pp.12-32]

A foreign key requires each non-NULL referencing value, or complete referencing combination under the relevant composite rule, to match an eligible referenced key. It does not require the sets of referencing and referenced values to be equal. A parent row can have no children, and several child rows can reference the same parent. Optional composite FKs need care about partial NULL combinations; do not infer an all-or-none rule without declaring one.

3.5 Updates and referential actions

INSERT adds rows; UPDATE changes qualifying rows; DELETE removes qualifying rows; DROP TABLE removes the table object. DELETE FROM Supplier leaves an empty Supplier table. Omitting WHERE in UPDATE or DELETE affects all rows. Adding a NOT NULL constraint is a schema operation, not an UPDATE of current values.

```
INSERT INTO Employee(empID, empName, deptID, salary)
VALUES (41, 'Mia Chen', 7, 65000);
UPDATE Employee SET salary = salary * 1.05 WHERE deptID = 7;
DELETE FROM Assignment WHERE projectID = 99;
ALTER TABLE Employee ALTER COLUMN empName SET NOT NULL;
```

An FK's configured action can reject a parent deletion or key update, cascade it, or set child references to NULL. SET NULL needs nullable columns. [L3A pp.23-38]

4. SQL: build the result one logical step at a time

4.1 Start with the result grain

Before writing SELECT, finish the sentence: **one output row represents ...** A student, a student-unit pair and one country summary have different grains. Identify which tables contain the requested facts, how they join, which rows qualify and whether duplicates are meaningful.

The logical teaching order is FROM/JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT and ORDER BY. The DBMS may execute an equivalent plan in another physical order. Understanding the logical order explains why WHERE cannot directly test a group count, why a missing join creates combinations, and why grouping at the wrong grain changes the answer. [L2B pp.7-27; L4B pp.4-14]

```
SELECT s.country, COUNT(*) AS students
FROM Student s
WHERE s.country <> 'AUS'
GROUP BY s.country
HAVING COUNT(*) >= 2
ORDER BY s.country;
```

WHERE removes Australian rows before grouping. GROUP BY forms one group per remaining country. HAVING retains countries whose group contains at least two rows. ORDER BY sorts the displayed groups. This does not count enrolments unless Enrolled has been joined in.

4.2 Joins and repeated entities

FROM R, S forms combinations of rows; a suitable join predicate retains the intended matches. An FK declaration does not automatically insert a join condition into a query. If Supplier and Product both contain SuppID, qualify that column when using both tables.

```
SELECT p.ProdID, p.SuppID
FROM Product p JOIN Supplier s ON p.SuppID = s.SuppID
WHERE s.Name = 'Heinz';
```

Without the equality condition, each Heinz supplier combines with every product. Without qualification, a reference to SuppID can be ambiguous. The bank flags a Canvas question whose published key describes the intended Cartesian-product effect even though its unqualified SELECT is ambiguous. Preserve the difference between the intended logic and executable SQL.

Use aliases to distinguish repeated uses of the same table. To compare employees with their managers, join Employee e to Employee m on e.managerID = m.empID. The two aliases represent different roles and can refer to different rows. A natural join matches **all** common attribute names, which is convenient only when those names express exactly the intended equalities. An unrelated same-named column can silently change the meaning.

4.3 Predicates, strings and dates

Use AND, OR and NOT with parentheses that express the intended logic. BETWEEN includes both bounds. IN lists alternatives; LIKE uses % for any-length text and _ for one character, while the taught PostgreSQL regular-expression operator ~ has different semantics. An equality predicate tests the whole value rather than a substring. String constants use single quotes, with a literal apostrophe doubled inside the string. Double quotes delimit SQL identifiers when needed; they are not the ordinary string delimiter. [L2B pp.14-21]

For dates, distinguish a stored DATE, a timestamp including time, and text that merely resembles a date. Use an unambiguous date literal such as DATE '2026-11-14' in new SQL. Date arithmetic and extraction can produce intervals or numeric components; follow the operator and datatype taught in the exercise rather than comparing a display-formatted string alphabetically. CURRENT_DATE denotes the current date in the database

context. Do not hardcode today's date if the task explicitly asks for a query that remains current.

A column alias labels an output expression; a table alias gives a name to an input table occurrence. SELECT expression AS label does not change the stored schema. Use ORDER BY with the required direction and tie-breaking columns when the requested output order matters.

4.4 Aggregates count different things

Expression	Meaning	Typical use
COUNT(*)	Number of result rows	Count activities or registrations
COUNT(x)	Number of rows with non-NULL x	Count supplied values or matched child keys
COUNT(DISTINCT x)	Number of distinct non-NULL x values	Count different students, instructors or years
SUM(x), AVG(x)	Sum or mean of non-NULL x values	Total hours or average grade
MIN(x), MAX(x)	Extreme non-NULL x value	Earliest date or greatest duration

For AssignmentMark(StudentID, UnitCode, AssignmentNumber, Score), COUNT(*) counts assessment records, not students. COUNT(DISTINCT StudentID) counts represented students. AVG(Score) gives an equally weighted mean over the non-NULL rows, which is not necessarily an average of each student's average. [L2B pp.31-34; L4B pp.21-25]

If the task asks for each unit and how many years it was offered, select uoSCode and COUNT(DISTINCT year), grouping by uoSCode. If it asks how many staff taught each unit when enrolment was at least 200, filter those offerings in WHERE before counting distinct instructor IDs. GROUP BY names attributes, not a table name or an English category.

4.5 Preserve zero counts with an outer join

An inner join removes a parent with no matching child. A left join preserves each left row and fills unmatched right columns with NULL. To count all activities, including those with no member who worked more than three days, put that condition in the join:

```
SELECT a.ActivityID, COUNT(w.MemberID) AS memberCount
FROM Activity a
LEFT JOIN WorksOn w
  ON w.ActivityID = a.ActivityID AND w.NumberDaysWorked > 3
GROUP BY a.ActivityID;
```

COUNT(w.MemberID) returns zero for the NULL-extended row. COUNT(*) would return one for an activity with no match. Putting w.NumberDaysWorked > 3 in WHERE would remove the unmatched row, defeating the requirement. This is one of the most useful small counterexamples for checking SQL. [L4B pp.26-31; revision paper A(ii)]

4.6 NULL and three-valued logic

NULL means unknown or inapplicable, not zero, an empty string or the string 'NULL'. Comparisons such as x = NULL and NULL = NULL are unknown. WHERE retains only true, so use IS NULL or IS NOT NULL. A condition x <> 5 does not retain rows where x is NULL.

For boolean combinations, false AND unknown is false; true AND unknown is unknown; true OR unknown is true; false OR unknown is unknown; NOT unknown remains unknown. SQL groups NULL values together for GROUP BY even though an equality comparison between them is unknown. This is a grouping rule, not evidence that NULL is an ordinary value.

Most aggregate functions ignore NULL inputs. COUNT(*) counts rows regardless of NULL; COUNT(x) excludes NULL x. Over an empty input, COUNT returns zero, while SUM, AVG, MIN and MAX return NULL. Use COALESCE(SUM(x), 0) if the requested report defines an absent total as zero. [L3A pp.13-15; L4B pp.21-25]

Why can joining Enrolled before COUNT(*) overcount students?

A student can contribute one joined row per enrolment. Count the Student table directly if no enrolment information is needed, use COUNT(DISTINCT sid), or use EXISTS when the purpose is only to test that a qualifying enrolment exists.

5. Advanced SQL: subqueries, sets and "every"

5.1 Identify the shape of a subquery result

A **scalar subquery** supplies one value; a multirow subquery supplies a set or bag to an operator that accepts it. A scalar comparison such as x = (subquery) is not a substitute for IN when several results are possible. EXISTS asks whether at least one row exists; its SELECT list is not used as returned data.

A **correlated subquery** refers to a variable from the outer query. Merely mentioning the same table, using aggregation or placing a subquery in WHERE does not make it correlated. Conceptually it is evaluated relative to each outer candidate row, though the optimiser may use a different equivalent execution strategy. [L4B pp.33-46]

```
SELECT s.sid, s.name
FROM Student s
WHERE EXISTS (
    SELECT 1 FROM Enrolled e
    WHERE e.sid = s.sid AND e.uos_code = 'ISYS2120'
);
```

The reference to s.sid creates correlation. One student is returned at most once from this Student scan even if several qualifying enrolment rows exist. A join could produce duplicates unless the join result is reduced appropriately.

5.2 Anti-joins, NOT IN and empty results

To find students with no enrolment, write NOT EXISTS with an equality linking the candidate student. A missing correlation makes the condition ask whether the entire Enrolled table is empty, which is a very different question.

```
SELECT s.sid FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e WHERE e.sid = s.sid);
```

NOT IN can behave unexpectedly when its subquery contains NULL. For example, 5 NOT IN (1, NULL) is unknown, not true. When NULL is possible, NOT EXISTS with the intended matching condition is usually the clearest way to express absence. If you use NOT IN, explain why the compared values are non-NULL. This is a logical caution, not a rule that every subquery must use EXISTS.

5.3 Maxima, ties and comparisons

If the request asks for all activities with the greatest Testing duration, compare with MAX over that same category. An ORDER BY with a single-row limit would discard ties. Applying the category filter only in the outer query compares Testing activities against the wrong population.

```
SELECT a.ActivityID FROM Activity a
WHERE a.ActivityType = 'Testing'
  AND a.Duration = (
      SELECT MAX(b.Duration) FROM Activity b
      WHERE b.ActivityType = 'Testing'
  )
ORDER BY a.ActivityID;
```

An aggregate without GROUP BY still returns a row on an empty table, so EXISTS(SELECT MAX(...) ...) is not a reliable non-emptiness test. Use EXISTS(SELECT 1 ...) instead.

ALL means the comparison must hold for every returned value; ANY or SOME means it holds for at least one. For an empty set, an ALL comparison is true and an ANY comparison is false. NULL can complicate these comparisons, so reason from the actual permitted data and the question's meaning.

5.4 UNION, INTERSECT and EXCEPT

UNION combines, INTERSECT finds common rows and EXCEPT subtracts rows. Inputs must have compatible output shapes. Ordinary forms eliminate duplicates; ALL variants use bag multiplicities. A row occurring m times on the left and n times on the right occurs m+n times in UNION ALL, min(m,n) times in INTERSECT ALL and max(0,m-n) times in EXCEPT ALL. [L4B pp.17-19]

To find students enrolled in both X and Y, intersect **student IDs**, not names. Then join or select names using the resulting IDs. Two different students may share a name; intersecting names can create a false apparent match. Likewise, a predicate uos_code = 'X' AND uos_code = 'Y' cannot hold for one ordinary enrolment row. The two required facts can occur in different rows.

5.5 Universal quantification: no missing requirement

"For every" is best translated as **there is no required item for which the candidate lacks a match**. Let RequiredUnit list the units all candidates must have taken.

```
SELECT s.sid FROM Student s
WHERE NOT EXISTS (
    SELECT 1 FROM RequiredUnit r
    WHERE NOT EXISTS (
        SELECT 1 FROM Enrolled e
        WHERE e.sid = s.sid AND e.uos_code = r.uos_code
    )
);
```

The inner NOT EXISTS identifies a missing match for one required unit. The outer NOT EXISTS rejects any student with such a missing unit. This handles additional unrelated enrolments without confusing them with required ones. If RequiredUnit is empty, every Student qualifies: there is no missing requirement. This is **vacuous truth** and should be checked against the intended business rule. [L4B pp.48-55]

An alternative compares the count of distinct required items matched by each candidate to the number of required items. It must restrict the numerator to the required set, avoid duplicate matches, and deliberately handle an empty required set. Counting all enrolments can accept the wrong students.

A student has taken X and Z; the required set is X and Y. Why does counting two enrolments give the wrong answer?

The count ignores which requirements were satisfied. The correct numerator is one distinct required unit, not two arbitrary enrolments. A double NOT EXISTS query detects the missing Y directly.

6. Relational algebra: derive the table you need

6.1 An algebra of relations

Relational algebra describes queries as compositions of operators. Each expression consumes and produces relations, so one result can be fed into the next operation. Read nested expressions from the inside outward. The mathematical model uses sets; projection removes duplicates and there is no inherent output order. SQL SELECT usually retains duplicates and can involve NULL, so a translation may require DISTINCT or other care. [L4A pp.8-30]

Operator	Meaning	Check before applying
σ condition (R)	Selection: retain qualifying tuples	Condition attributes must still be present
π attributes (R)	Projection: retain named attributes and remove duplicates	All requested attributes must exist
R × S	Cartesian product: every pair	Rename ambiguous same-named attributes as needed
R ⋈ condition S	Conditional join: product restricted by condition	State the intended equality or other predicate
R ⋈ S	Natural join on all common attribute names	Check every common name
R ∪ S, R ∩ S, R - S	Union, intersection, difference	Compatible relation schemas
ρ newName (R)	Rename a relation or its attributes	Needed for self-joins and compatible set operations
R ÷ S	Division: values related to every required S tuple	Identify candidate and requirement attributes

This guide sometimes writes explicit English names such as PROJECT_name(...) alongside the symbols. These are explanatory notations; use the notation accepted in the unit when writing an exam answer.

6.2 Projection cannot restore a discarded attribute

For Car(regno, model, year, owner), the registration numbers of pre-2002 cars are:

```
PROJECT_regno(SELECT_year<2002(Car))
Equivalent symbols: pi_regno(sigma_year<2002(Car))
```

First retain rows using year, then project regno. Projecting regno first removes year and makes the later year condition undefined. Selection filters rows; it does not choose columns. Projection selects columns; it does not use a Boolean condition as an attribute list. [L4A pp.21-24]

6.3 Joins, absence and "both"

For Person(license, name, address) and Car(regno, model, year, owner), find the owner name of NSW314 by joining on owner = license, selecting the registration and projecting name. A natural join would not know that owner and license mean related identifiers when their names differ.

```
PROJECT_name(
  SELECT_regno='NSW314'(
    Person JOIN_owner=license Car
  )
)
```

For students enrolled in DBMS, join Student to Enrolled on sid and Enrolled to UnitOfStudy on uos_code; filter title = 'DBMS'; project name. For students enrolled in no unit, subtract PROJECT_sid(Enrolled) from PROJECT_sid(Student). Subtract compatible IDs, then retrieve names if needed. [L13 p.33]

For suppliers of **red or green** products, select colour='red' OR colour='green' before joining to Catalog and Supplier. For suppliers of **both red and green** products, construct the red supplier-ID set and the green supplier-ID set and intersect them. A single product tuple cannot generally have colour equal to both values. Intersecting supplier names can confuse distinct suppliers with the same name. [L13 p.34]

6.4 Division and equivalence

If Takes(sid, ucode) records all taken units and Required(ucode) is the required set, Takes ÷ Required returns the sid values in the dividend's candidate domain that match every required unit. The SQL double NOT EXISTS version in Chapter 5 can deliberately choose candidates from Student instead. These differ on an empty requirement set if students without any Takes row are absent from the dividend. State the candidate domain.

Selection can often be pushed before a join if it refers only to that input's attributes. For example, filtering red Product rows first reduces intermediate data while preserving the answer of an ordinary inner join. Projection can often be pushed down too, but retain join attributes and anything needed later. Such transformations justify query optimisation; they do not permit changing what the query asks. Outer joins and NULL require additional care, so do not apply inner-join rewrite rules indiscriminately. [L4A pp.31-40]

Given Deliverer(postcode, emp_id, name) with emp_id 103 in two rows, what does projection on emp_id produce?

One occurrence of 103. In the mathematical algebra, a relation is a set. SQL SELECT emp_id can show it twice; SELECT DISTINCT emp_id matches the set projection.

7. Functional dependencies, keys and normalisation

7.1 Redundancy is about repeated facts

A wide table can mix facts with different determinants. If each employee belongs to one department, repeatedly storing that department's phone on each employee row risks an **update anomaly**: one copy changes while another does not. An **insertion anomaly** can prevent recording a new department without inventing an employee. A **deletion anomaly** can lose the department's details when its last employee is deleted. Storage waste matters, but inconsistent or unrepresentable facts are the larger concern. [L6 pp.5-15]

Normalisation evaluates a schema using known domain dependencies. First check **adequacy**: can the schema represent the required facts? Then reduce harmful redundancy. Splitting tables arbitrarily can lose associations even when every original column appears somewhere.

7.2 Read the direction of an FD exactly

An FD X -> Y means: in every legal instance, any two tuples equal on **all** attributes of X must be equal on **all** attributes of Y. A violation needs a pair that agrees on the complete determinant X and differs on at least one dependent Y attribute. It does not require X values to be unique, and it does not imply Y -> X.

For EmpID -> DeptID, several employees may share one department. Repeated EmpID is permitted in a larger relation if DeptID stays consistent. Observing an instance can disprove an FD by showing a counterexample, but cannot establish that it must hold in every future legal state. Domain semantics provide the constraint. [L6 pp.17-24]

An FD is **trivial** when its RHS is a subset of its LHS. A **superkey** determines all attributes. A **candidate key** is a minimal superkey: remove any one attribute and it stops being a superkey. Minimal means no proper subset is a key, not necessarily that it has the smallest cardinality among all candidate keys. One candidate key may be selected as the primary key.

7.3 Attribute closure: the repeat-until-stable method

To calculate X+, start with X. Repeatedly apply any FD whose **entire LHS** is already present, adding its RHS. Recheck previously unusable FDs after new attributes appear. Stop when no FD adds anything. X -> Y follows from the given FDs exactly when Y is contained in X+.

Worked example. Let R(A,B,C,D,E,F,G,H,I), with AC -> GI, B -> D, D -> C, E -> D, F -> E and G -> D. Calculate (AE)+:

Step	Closure so far	Reason
Start	A,E	Given determinant
1	A,D,E	E -> D
2	A,C,D,E	D -> C
3	A,C,D,E,G,I	AC -> GI
Stop	A,C,D,E,G,I	G -> D adds nothing; no rule yields B, F or H

AE is not a superkey because B, F and H are missing. CE -> D is implied by E -> D, but CE -> F is not: F -> E cannot be reversed. This is the shared calculation in the Week 7 quiz. [L6 pp.26-35; Canvas Week 7 Q2-Q4]

7.4 Prove a key, including minimality

For the same relation, attributes A, B, F and H never appear on any RHS. They must occur in every superkey under these FDs, because closure cannot generate them. (ABFH)+ adds E from F, D from B or E, C from D, then G and I from AC; it is all of R. Therefore ABFH is a candidate key. This argument also explains why a proposed key missing H cannot work.

An attribute absent from every RHS is mandatory in all keys, but the mandatory set is not always sufficient: additional attributes may be needed to start a cycle of dependencies. Always compute its closure. To show a particular superkey is not a candidate key, exhibit a proper subset that remains a superkey. [L6 pp.23-25, 36-43]

Second worked example. For R(A,B,C,G,H,I), with A -> B, A -> C, CG -> H, CG -> I and B -> H, (AG)+ becomes ABG, then ABCG, then ABCGH and ABCGHI. A+ is ABCH; G+ is G. Thus AG is a candidate key. The argument needs both coverage and minimality. [L13 p.39]

Does a superkey always become a candidate key by removing exactly one attribute?

No. It may already be a candidate key, or it may contain several unnecessary attributes. A superkey contains some candidate key, possibly itself, but an exact one-attribute removal is not guaranteed.

7.5 BCNF: every nontrivial determinant is a superkey

A relation is in **Boyce-Codd Normal Form** when for every nontrivial FD X -> Y that holds on the relation, X is a superkey. You can disprove BCNF using one nontrivial FD whose determinant's closure misses an attribute. Checking that the primary key determines the other columns is not enough; that is already true of a key.

For S(A,B,C,D,E), with A -> CE, D -> B and E -> DA, both A and E determine all attributes. D+ = DB, so D -> B violates BCNF. Decompose into S1(D,B) and S2(A,C,D,E). S1 is BCNF because D is its key. In S2, A and E remain keys and the projected dependencies do not introduce a nonkey determinant. [L13 p.42]

When analysing a decomposed relation, use dependencies projected onto its attributes. An implied FD can survive even if the intermediate attribute used to infer it is outside that subrelation. Do not mechanically discard every original FD mentioning a removed attribute and assume the result is complete.

7.6 Lossless join and dependency preservation

A decomposition is **lossless** when joining its projections reconstructs exactly the original relation in every legal instance: no lost tuples and no spurious combinations. For a binary decomposition R1,R2 covering R, the FD criterion is that their intersection determines all of R1 or all of R2. It need only determine **one** component, not both.

For S1(D,B) and S2(A,C,D,E), the intersection is D and D -> DB. The decomposition is lossless. To split on a violating FD X -> Y, the standard construction uses R1 = X union Y and R2 = R minus (Y minus X). Keep X in both parts; it is the join connection.

Dependency preservation is a separate property: the union of the FDs enforceable within the component relations should imply the original FDs. This permits checking constraints locally without joining tables. A lossless decomposition can fail to preserve dependencies, and a dependency-preserving decomposition can be lossy. BCNF decomposition always permits a lossless result, but does not always preserve every dependency. [L6, normal forms and decomposition sections]

Counterexample to confusing the properties. Let R(A,B,C), F = {AB -> C, C -> B}. Decompose into CB and AC. It is lossless because common C determines CB. Both components are BCNF, but AB -> C is not implied by the local projected dependencies: starting with AB, C cannot be generated. Enforcing only C -> B therefore misses the original AB constraint.

8. Security, privacy and integrity mechanisms

8.1 Identify the property and the policy

Confidentiality prevents unauthorised disclosure. **Integrity**, in the security sense, prevents unauthorised changes. **Availability** lets authorised users perform permitted operations when needed. These differ from semantic integrity constraints, which specify allowed data values or relationships. CHECK(salary >= 0) can reject an invalid value, but it does not decide which employee is allowed to change salaries. [L7 pp.6-8]

A **policy** states who should be allowed to do what under which conditions. A **mechanism** implements that policy. Authentication establishes an identity; authorisation checks that identity's privileges. A correct password check does not imply permission to access every record. Data minimisation reduces exposure by avoiding unnecessary collection and retention; it complements access control rather than replacing it.

8.2 GRANT, roles and delegation

```
CREATE ROLE payroll;
GRANT USAGE ON SCHEMA public TO payroll;
GRANT SELECT(empID, empName), UPDATE(salary)
    ON public.Employee TO payroll;
GRANT payroll TO mia;
REVOKE payroll FROM mia;
```

Column privileges can restrict which columns are read or modified. UPDATE(salary) permits salary changes across rows; it does not restrict them to one department. A row-dependent rule needs an appropriate further mechanism, such as the course's view-based approach. Schema USAGE and connection to the correct database are distinct prerequisites from table privileges. [L7 pp.13-21]

WITH GRANT OPTION lets a recipient pass the specified object privilege to others. GRANT SELECT ON T TO X WITH GRANT OPTION does not let X grant UPDATE on T or SELECT on every other table. Privileges can arrive through several paths or roles; removing one path may leave another. Revocation needs to account for dependent grants and the relevant platform's cascade/restrict behaviour. Ownership and powerful administration privileges require separate consideration.

Roles group permissions and can be granted to several users. A user can belong to several roles; there is no course rule limiting membership to two. PostgreSQL represents users as roles with login-related properties. Do not confuse role membership with a fixed per-table permission or an application-only account.

8.3 Views provide restricted interfaces

A normal view stores a query definition, not an independent saved copy of its rows. Its logical content reflects the underlying tables when queried. A view can select columns, filter rows or expose aggregates. Grant access to the view while withholding inappropriate access to base tables, or the user may simply bypass the restriction. [L7 pp.28-41]

```
CREATE VIEW activity39_members AS
SELECT DISTINCT m.Name, m.Role
FROM Member m JOIN WorksOn w ON m.MemberID = w.MemberID
WHERE w.ActivityID = 39;
GRANT SELECT ON activity39_members TO joe;
```

This supplies the requested names and roles. It does not grant modification rights. The complete policy also requires that Joe lacks another grant or role permitting the prohibited base-table operations. A view alone does not remove permissions that already exist. An aggregate view may still reveal individual information in very small groups or through comparison; consider what the released summaries actually disclose.

Some simple views are updatable, but joined or aggregate views can lack a unique mapping back to base-table changes. Raising a department's average salary does not uniquely specify whose salaries should change. WITH CHECK OPTION prevents supported view updates from producing rows outside the view's defining condition; it does not solve every ambiguity of view update.

8.4 Constraints, triggers and stored logic

Declared PK, FK, NOT NULL and CHECK constraints centralise rules so that every relevant writer is checked. Assertions express more general conditions in the SQL model, but are not supported uniformly; do not present CREATE ASSERTION as an executable PostgreSQL solution. Cross-row or transition rules may require triggers or correctly coordinated application logic. [L7 pp.43-62]

A **trigger** runs automatically on a specified event, with an optional condition and an action. It can support auditing or enforce a rule relating old and new states. A **stored procedure** is application logic stored and executed inside the DBMS, invoked by a client. It is not a program running in the browser. A stored routine can reduce round trips and centralise logic, but its result interface, privileges and transaction behaviour depend on the platform and routine type.

The course uses "stored procedure" broadly in discussing stored logic and trigger actions. For actual PostgreSQL code, distinguish procedures from functions: do not infer that every stored routine returns a whole result set or that all can be invoked in the same way. The core exam distinction is server-side stored logic versus host-language application code. [L7 pp.63-70; L8A pp.44-46]

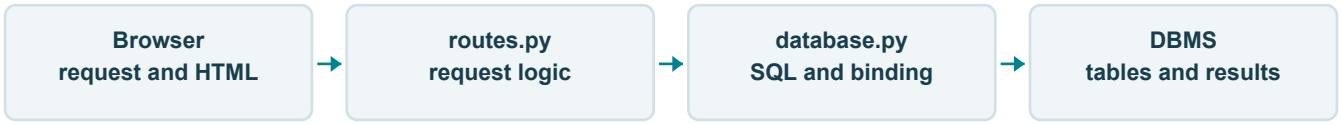
A deferrable constraint may be checked at a later allowed point in a transaction when configured appropriately; "deferrable" does not mean "never checked." Not every SQL constraint type on every platform supports deferral. Distinguish the general course mechanism from details of runnable vendor DDL.

A manager may change Salary for all employees, but nothing else. Must you create a view?

Not for that column-only policy: GRANT UPDATE(Salary) ON Employees TO manager expresses the modification permission. If the policy limits the rows to JobCategory='Secretarial', a plain column grant is insufficient.

9. Database-backed applications and their security

9.1 Trace a request across three layers



The browser sends an HTTP request to the application server. A Flask route reads path or form values, checks application conditions, calls database functions and chooses a response. Database code uses a DBMS connection and cursor to execute SQL. Results become Python values, then a template renders the response, commonly HTML, which the browser displays. SQL is not normally sent to the end user's local database to render a page. [L8A pp.5-14; Lab8 pp.5-7]

In the supplied scaffold, web_app.py starts the application, routes.py connects URLs and request data to actions, database.py constructs and executes SQL, templates contain presentation markup, and config.ini holds connection configuration. These roles explain why "a method in database.py" is the published answer to the recurring code-location question. A template is important for following the input and output, but is not the primary query-construction location.

9.2 Connection, statement and cursor are different

A **connection** or session is an ongoing communication context with the DBMS, including authentication and transaction state. A **parameterised statement** is a fixed SQL structure plus bound values. A **cursor** provides an interface to execution and results, allowing the application to fetch rows. It is not the user identity, a primary-key lookup without SQL or the same object as a connection. [L8A pp.17-38]

In a **call-level interface**, normal host-language code calls a database library with SQL strings and parameters. **Embedded SQL** introduces special SQL syntax into host-language source and typically needs special processing. A call-level program is not defined by mixing SQL keywords directly into the host language grammar.

```
cur = conn.cursor()
cur.execute('SELECT empName FROM Employee WHERE empID = %s',
            (employee_id,))
row = cur.fetchone()
# row may be None when no matching record exists.
cur.close()
```

The placeholder style here follows the supplied pg8000 scaffold. Placeholder syntax varies by driver. A one-element Python tuple needs its comma. Bind the value separately; do not apply Python string interpolation to fill the placeholder. Handle database exceptions, commit or roll back as needed, and release resources even on failure.

9.3 Find injection by following untrusted input to SQL

SQL injection occurs when input changes the grammatical structure or meaning of a query because code includes it in SQL text. It is not simply any database error, leaked error message or stolen password. Concatenation, f-strings and formatting around user-controlled fragments deserve inspection. Parameter binding protects **values**; it does not turn an arbitrary column name or operator into a safe identifier. [L8B pp.10-14]

The source of the local Week 8 ZIP shows these concrete paths:

File and function	Observed source pattern	What a reviewer should establish
routes.py: search_users_byname	Passes request.form['searchfield'] and ['searchterm'] to the database helper	Treat both values as untrusted, regardless of the HTML widget
database.py: search_users_customfilter	Interpolates attributename and filtertype into an f-string; passes filterval separately	Allowlist identifiers and operators; retain value binding
database.py: list_users_equifilter	Interpolates attributename; binds filterval	Determine whether each caller supplies a fixed trusted name
database.py: update_single_user	Interpolates userid into WHERE; builds SET fragments	Bind userid and review every dynamic fragment and caller
database.py: delete_user	Interpolates userid; passes an empty parameter tuple	An execute call with parameters is insufficient if input is already in SQL text
database.py: check_login and add_user_insert	Fixed SQL with values supplied in a separate tuple	This is a suitable value-binding pattern, subject to other security requirements

This is source inspection, not a report of live exploit testing. The ZIP also contains ordinary correctness issues, so not every failure is evidence of injection. The task is to explain where input can affect SQL structure and which checks prevent that path.

A safe design maps a small public choice to fixed internal SQL identifiers and operators, then binds data values. For example, map 'first_name' to a literal firstname column; reject unknown choices. For a contains search, put '%' + term + '%' into the bound value. Do not assume a placeholder can stand for a table name. A dropdown changes the browser interface; a crafted HTTP request can still send another value, so enforce the allowlist on the server.

Is cursor.execute(f'SELECT * FROM users WHERE userid={x}', ()) safe because it has a second argument?

No. x has already been inserted into SQL text before execute runs. Replace it with a value placeholder and pass x as the bound parameter. Review the entire construction path rather than counting execute arguments.

9.4 Accounts and other application threats

Distinguish operating-system accounts, DBMS accounts and application accounts stored in the application's user tables. A browser user may log into the application while all its database work runs under one configured DBMS account. The application must then enforce per-user permissions itself, and the DBMS account should have only the permissions it needs. [L8B pp.5-9]

Store application passwords using an appropriate verification design rather than plaintext, and avoid exposing them through SELECT * or logs. Protect DBMS connection configuration from public repositories and downloads. These concerns are distinct from injection.

Persistent XSS occurs when stored attacker content later executes in a browser. Escape output for its rendering context and control safe-HTML bypasses. SQL parameter binding addresses a different interpreter and does not prevent XSS. Keep raw DBMS errors in controlled diagnostics rather than user responses. [L8B pp.15-19]

10. Storage, indexing and query processing

10.1 Why physical organisation matters

Database records live on persistent storage and are transferred in pages or blocks. Access to storage is much slower than access to memory, so I/O can dominate a query. A **buffer pool** caches pages in RAM and tries to make needed pages available when used. It does not replace persistent storage, guarantee protection from application mistakes or make dirty pages automatically durable. [L9 pp.8-18]

A **heap file** has no ordering by a chosen search key. Insertions can be relatively cheap, but a query without a helpful index may scan every page. Deletions and variable-sized records mean it need not be densely packed at every moment. A sorted file helps searches matching its ordering, but updates can be more costly. SQL output still needs ORDER BY when order matters. [L9 pp.19-29]

10.2 Classify indexes on independent axes

Property	Question it answers	Consequence
Tree versus hash	How are index entries searched?	Tree supports ordered ranges; hash mainly equality
Clustered versus unclustered	How are matching data records physically located?	Clustering improves locality for many matches
Single versus composite key	Which ordered attribute sequence is indexed?	Composite order controls useful prefixes
Unique versus nonunique	Can several rows share a search key?	Uniqueness is distinct from physical clustering
Covering for a query	Are the needed values available from the index?	May avoid fetching base records in the course model

A search key need not be a primary key or a unique key. Several rows can have the same indexed department or date. Many unclustered indexes can coexist, while one table can have only one physical clustering order at a time. PostgreSQL's course example uses an unordered heap by default and a one-time CLUSTER operation; a PK should not be assumed to maintain physical clustering. [L9 pp.33-46]

10.3 Equality, ranges and composite order

A B+-tree stores entries in search-key order and supports equality and range lookup. A hash index can locate an equality bucket but does not order values for efficient range navigation. A hash index on d is useful for d=26 even if the query also filters a range of c; the c condition is then tested on candidate records. A range on d alone is not its main strength.

An index on (a,b) is lexicographically ordered: first by a, then by b within each a. Equality a=5 followed by a range on b is a natural selective use. Rows with a given b value are not necessarily adjacent across different a values. For a query on d with an index (d,c), d is a useful leading key; c without a restriction on d lacks that straightforward prefix lookup. [L9 pp.42-43; L10 pp.30-34]

Covering needs two checks. Ask whether the index contains every value needed for output and filters, then whether its order supports the intended efficient search. The course examples emphasise filter attributes as a leading prefix. In broader implementation terms, an index can sometimes supply all needed columns through a

scan even without a selective leading-key lookup. Do not confuse physical availability with an efficient seek, and do not assume a primary key is automatically stored in every secondary index.

10.4 Compare access costs with explicit assumptions

A table scan reads B table pages and checks their records. With a tree of lookup height h and m matches, a clustered range scan can cost roughly h plus the few pages containing those adjacent records. An unclustered lookup may require roughly h plus as many data-page reads as matching records in the cold-cache worst case. Actual cost depends on repeated page hits, caching, selectivity, row size and implementation; these are reasoning models, not measured runtimes. [L10 pp.12-29]

Worked example. Suppose the table has 10,000 pages, a tree lookup needs three page reads, and 400 matching records fit on four adjacent data pages. A clustered lookup might need roughly 3+4 page reads. If the same matches are scattered across 400 cold pages, an unclustered plan might need roughly 3+400. Both can beat 10,000, but if most records match, a sequential scan may be preferable. Do not answer "an index is always faster."

10.5 Optimisation versus tuning

Query optimisation chooses an execution plan for a particular query from logically equivalent alternatives. It aims for a low estimated cost while preserving meaning; it does not choose whichever SQL statement gives the "most accurate" answer. The parser, translator, optimiser and executor have different responsibilities. [L10 pp.4-8]

Database tuning chooses physical design and other arrangements for a workload: which indexes to create, what to cluster, and what the read/write tradeoffs are. Each extra index consumes storage and must be maintained on relevant changes. Favour changes that help important frequent or expensive requests, and account for costs to insertions, updates and deletions. [L10 pp.35-43]

For WHERE Role='Tester' AND Joined<'2022-01-20', a tree on (Role, Joined) can use equality on its leading attribute then a date range. Adding needed output columns can make a suitable covering design in the course model, but increases index size and update cost. This is a justified proposal, not a promise of a particular speedup.

Does a clustered index on (a,b) group every row with the same b together?

No. It groups by a first. Equal b values can be split across many a groups. A clustering order led by b is the appropriate answer to that locality requirement.

11. Transactions, ACID and concurrency

11.1 A transaction is a logical unit of work

A transaction groups one or more operations that should be treated as one task. A bank transfer must debit one account and credit another together. COMMIT requests completion; ROLLBACK cancels the transaction's effects. A system can also abort work because of errors or conflicts, so the application must handle failure rather than assume a commit request always succeeds. Autocommit treats individual statements as separate transactions and can therefore split a business operation that needs several statements. [L11 pp.5-17]

Property	Meaning	Transfer example
Atomicity	All effects of the transaction occur, or none do	No committed debit without its corresponding credit
Consistency	A correctly written transaction preserves the required valid-state rules	Total money and account rules hold at successful completion
Isolation	Concurrent work has the promised separation; serializable isolation gives a serial-equivalent outcome	Concurrent transfers do not invalidate reasoning based on each transfer
Durability	Committed effects survive the failures covered by the DBMS guarantee	A committed transfer is retained after restart

Atomic does not mean one statement, indivisible CPU execution or "the program reached its final line." Consistent does not mean that every intermediate state must satisfy every business rule. The programmer must write the task to preserve its intended invariants, while the DBMS enforces declared constraints at their configured checking points. Some constraints are immediate; others can be deferred when the platform supports that. [L11 pp.18-26]

11.2 Consistency is an obligation of the application

A business rule may not be declared in the schema. If a transaction starts in a valid state, its code should either establish another valid state and commit, or roll back. If a task decreases a stock quantity and later discovers that a cross-store minimum would be violated, it must undo its earlier changes. Committing just the first part breaks the rule even if every SQL statement was syntactically legal.

Serializability supports **local-to-global reasoning**: if each whole transaction preserves consistency in isolation, and the DBMS makes their combined execution equivalent to a serial order, the combined result preserves those rules. Weaker isolation can invalidate this argument. Serializable does not mean transactions literally run one at a time, nor that they can never observe earlier committed work. [L11 pp.25-52]

11.3 Recognise the anomaly from its mechanism

Dirty read. T2 reads a value written by T1 before T1 commits. If T1 aborts, T2 has used a value that should never have become permanent. The key issue is reading uncommitted work, not simply two reads occurring near each other.

Lost update. Two tasks read a value and separately write replacements based on those old reads. Stock is 10. Each reads 10; one writes 9, the other writes 8. If the intended changes are -1 and -2, the final result should be 7, but a replacement overwrites another change. Single-statement arithmetic updates and appropriate

transaction control can change the concurrency behaviour; do not assume every concurrent UPDATE creates this anomaly.

Inconsistent read. A task reads related items from different logical moments and reaches a conclusion that would not arise from a consistent serial view. **Write skew** can occur when concurrent tasks read overlapping conditions but update different rows, collectively violating a rule even though each task's own view seemed safe. These are distinct from one transaction failing to check its business rules. [L11 pp.41-49, 66-67]

Worked example: two doctors on call. A rule requires at least one doctor on call. Doctor A's transaction sees B on call and turns A off; B's transaction sees A on call and turns B off. The writes affect different rows, so avoiding a direct write-write conflict is not enough. Appropriate isolation or a correctly designed coordination mechanism must protect the shared condition.

11.4 Recovery, logging and locks

The course describes a log that records changes. Undo can remove effects of aborted or incomplete transactions; redo can restore committed effects that were not yet reflected in persistent data pages at failure time. This supports atomicity and durability. A buffer pool alone cannot provide these guarantees, because memory is volatile and dirty pages may be written at different times. [L11 pp.54-58]

Locks coordinate conflicting access. Holding necessary locks until transaction completion can prevent unsafe interleavings, but can also cause waits and deadlocks that the system must handle. Logging, concurrency control and application-level checks solve different parts of reliability; none eliminates the need for the others. The lecture's distributed-commit discussion explains why a task spanning several databases needs coordination rather than unrelated local commits.

11.5 Isolation levels are deliberate tradeoffs

The lecture discusses READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ and SERIALIZABLE. Lower isolation can improve concurrency but allows more anomalies; exact guarantees and defaults vary by platform. The supplied slides identify PostgreSQL's default as READ COMMITTED. Treat this as the course's platform context rather than assuming every database has that default. [L11 pp.60-69]

Use SERIALIZABLE when the correctness argument requires a serial-equivalent outcome, and handle a rejected transaction appropriately. READ COMMITTED prevents reading another transaction's uncommitted changes, but successive statements can still see different committed states. A snapshot or repeatable view alone should not be assumed to prevent every cross-row invariant failure. The important skill is to identify which concurrent observation can break the particular business rule.

A transfer is implemented as two autocommit UPDATE statements. What can go wrong if the second fails?

The first statement may already be committed, so rolling back the second cannot undo it. Put the logically connected operations into one transaction and handle failures for the whole unit. This is an atomicity problem even with only one active user.

12. Analytics and data governance

12.1 OLTP and OLAP have different workloads

OLTP supports operational work: frequent short requests, current data and small sets of records, often with updates. **OLAP** supports analysis: larger scans, aggregates, historical comparisons and exploration. An order-entry task is OLTP; comparing sales across product, state and quarter is OLAP. They can share underlying facts while benefiting from different schemas and physical organisations. [L12 pp.4-12]

A **data warehouse** integrates selected data from operational systems, often with historical content and a schema chosen for analysis. Keeping analysis on a separate server can reduce interference with operational response times and support different physical choices. This does not mean every warehouse is updated continuously or must copy each source schema exactly.

12.2 ETL is a semantic integration task

Extract obtains source data, **transform** cleans and reconciles it, and **load** stores it in the warehouse. Integration can require matching identifiers, agreeing on units and time conventions, reconciling different definitions, handling missing values and retaining history. Metadata records sources, meanings and transformations so that later users can interpret results. Incremental updates reduce repeated work but must correctly identify changes. [L12 pp.13-19]

If one system records gross sales and another records net sales, simply concatenating their numbers produces a misleading total. A technical type conversion is not a substitute for agreeing on the meaning of the measure. State the fact table's **grain** before aggregating: one transaction line, one daily product-store summary or one shipment.

12.3 Facts, dimensions and navigation

A **fact table** records measures at a defined combination of dimension values. Sales amount or quantity is a measure; product, market and time are dimensions. Dimension tables describe those dimensions. A **star schema** connects a central fact table to dimension tables. A dimension hierarchy, such as city -> state -> country, supports different levels of aggregation. [L12 pp.21-30]

Operation	What changes	Example
Roll up	Summarise at a coarser hierarchy level	Sum state sales into country sales
Drill down	Inspect a finer level	Country totals to states, then cities
Slice	Restrict a dimension to a selected value	Sales for 2026 only
Dice	Restrict a subspace using several criteria	Selected products, states and months
Pivot	Change displayed grouping dimensions or orientation	Product-by-state versus product-by-quarter

When rolling up averages, do not blindly average subgroup averages if subgroup sizes differ. Use the underlying totals and counts when those are available. A two-person group's mean and a hundred-person group's mean should not automatically receive equal weight.

12.4 CUBE and window concepts

GROUP BY CUBE(a,b) produces aggregates for (a,b), (a), (b) and the empty grouping set: four grouping combinations. More generally, n dimensions give 2^n grouping sets. It adds marginal and grand totals **alongside** ordinary detailed groups. It does not mean the output must be a physical cube or that running time is the cube of the row count. [L12 pp.34-38]

```
SELECT market_id, product_id, SUM(sales_amt)
FROM Sales
GROUP BY CUBE(market_id, product_id);
```

NULL in a subtotal dimension represents aggregation over that dimension in this output convention. Real missing values can also be NULL, so interpretation needs attention. GROUPING SETS syntax is explicitly excluded in the provided lecture and is not required here.

A **window aggregate** calculates relative to each reference row, using a partition, an order and a frame. GROUP BY normally collapses a group to one row; a window calculation can preserve one output row per input row while adding a moving average or rank. The unit asks for this conceptual distinction rather than detailed window syntax. [L12 pp.40-45]

In **ROLAP**, warehouse data remains in relational tables accessed with SQL; a cube is a conceptual analysis view. **MOLAP** uses a specialised multidimensional storage approach. A visual chart or cube display does not by itself tell you which storage model is underneath. [L12 p.46]

12.5 Lakes, lakehouses and governance

The lecture contrasts a warehouse's organised, selected relational data with a **data lake**, which retains varied data for later interpretation, and a **lakehouse**, which combines flexible data storage with some warehouse-style management. Treat the simplified data-lake limitations in the lecture as a motivation for governance, not a claim that every contemporary lake platform has identical capabilities. [L12 pp.48-52]

Data governance assigns decision rights, responsibilities and processes for handling data across the organisation. It includes knowing where data is kept, who can authorise use, how quality and lineage are checked, how long it is retained, and who is accountable when something goes wrong. Security controls are part of governance, but governance also concerns appropriate use and authority. [L12 pp.54-60]

The course's data-sovereignty discussion distinguishes **authority** to decide from **capability** to exercise and enforce that decision. Physical server location is only one factor; organisational control, replication, provider obligations, auditability and affected communities' rights also matter. These are course concepts, not legal advice about a particular jurisdiction.

The **CARE principles** for Indigenous data governance are Collective Benefit, Authority to Control, Responsibility and Ethics. Their focus includes people, communities, knowledge and non-human relations. A technically accessible dataset is not automatically appropriate to reuse or make open. Explain whose authority and interests matter, which benefit is intended and how accountability is demonstrated. [L12 pp.61-63]

Is a warehouse's data cube necessarily a non-relational physical structure?

No. ROLAP stores the information in relational tables and uses SQL. The cube can be a way to organise questions and visualise results. MOLAP is the distinct specialised storage approach.

13. Integrated worked methods

13.1 From an English request to a checked SQL answer

Use the activity schema in the revision paper: Member(MemberID, Name, Joined, Role, Location), Activity(ActivityID, ActivityType, Leader, Duration), WorksOn(MemberID, ActivityID, NumberDaysWorked). The pair (MemberID,ActivityID) is WorksOn's key; Leader references Member. "How many activities longer than two days are led by a Tester?" asks for activities, not members or work assignments.

```
SELECT COUNT(*) AS activityCount
FROM Activity a JOIN Member m ON a.Leader = m.MemberID
WHERE a.Duration > 2 AND m.Role = 'Tester';
```

The join has at most one member match per activity because MemberID is a key. Joining WorksOn unnecessarily can multiply each activity by its participants. A useful verification dataset contains an activity with several workers, one with no leader, one of duration exactly two and one led by a non-Tester. It should challenge the boundaries rather than merely provide a happy example.

For total hours on activity 57, aggregate NumberDaysWorked, not Duration. Duration describes the activity; days worked describes each member's contribution. Multiply the sum by eight. If no work rows should mean zero, use COALESCE; if the question treats unknown totals differently, state that interpretation.

```
SELECT COALESCE(SUM(NumberDaysWorked), 0) * 8 AS totalHours
FROM WorksOn WHERE ActivityID = 57;
```

Useful counterexamples. For joins, include two children per parent. For all-parent counts, include a parent with no children. For maximum questions, include a tie. For anti-joins, include a NULL when permitted. For "every," include an empty required set and unrelated matches. These checks catch semantic errors that a query merely executing would not reveal.

13.2 A full BCNF decomposition with proof

Let Contracts(contractID, supplierID, projectID, deptID, itemID, quantity, value) have contractID -> all other attributes, supplierID deptID -> itemID and projectID -> supplierID. contractID is a candidate key. Neither supplierID deptID nor projectID determines all attributes, so both listed nontrivial dependencies violate BCNF. [L13 p.43]

Split first on supplierID deptID -> itemID:

```
R1(supplierID, deptID, itemID)
R2(contractID, supplierID, projectID, deptID, quantity, value)
```

The common supplierID,deptID determines R1, so this split is lossless. R1 is BCNF under its projected dependencies. R2 still violates BCNF through projectID -> supplierID. Split it into:

```
R3(projectID, supplierID)
R4(contractID, projectID, deptID, quantity, value)
```

The common projectID determines R3, so the second split is lossless. R3 is BCNF with key projectID. R4 is BCNF under the projected dependencies, with contractID as key. Therefore R1,R3,R4 form a lossless BCNF decomposition. The original dependencies can also be enforced or inferred from these components: contractID determines its R4 attributes, then supplierID through R3 and itemID through R1. State these arguments rather than asserting that any collection of smaller tables is automatically normalised.

13.3 DoctorsInfo: do not invent a missing dependency

The supplied revision paper gives PatientID -> Age,PostCode,City; PostCode -> City; ProvidorNo -> Specialty; VisitID -> VisitDate,PatientID. It does **not** say (VisitID,ProvidorNo) -> Diagnosis. Calculate that pair's closure by adding VisitDate and PatientID, then Age, PostCode and City, then Specialty. Diagnosis remains missing. Therefore the pair is not a superkey under the given FDs. [Revision paper pp.4-5]

A candidate key is (VisitID,ProvidorNo,Diagnosis): VisitID, ProvidorNo and Diagnosis do not appear on any RHS, so all are necessary, and their combined closure contains every attribute. The apparent medical meaning or the current sample's uniqueness does not authorise adding an FD to the problem.

For a binary lossless split, isolate Provider(ProvidorNo,Specialty) and put all other attributes plus ProvidorNo in VisitDetail. Common ProvidorNo determines Provider. Provider is BCNF. VisitDetail still contains dependencies such as PostCode -> City with a nonkey determinant, so it is not yet BCNF. The question asks for **two relations**, so do not silently substitute a complete five-table normalisation unless you label it as an optional extension.

13.4 A security explanation with concrete mechanisms

For an audience of new Flask developers, start with the request path and name the files and functions. Then explain the vulnerability condition: input reaches query structure, rather than a bound data parameter. Describe the repair: fixed SQL structure, server-side allowlists for variable identifiers/operators, separate value binding and suitable authorisation. Finish by identifying verification limits.

For a DBA, a view-and-grant answer should name the permitted rows and columns, show the view definition, grant the precise privilege and account for bypass access. For a nontechnical manager, lead with the policy and consequence before implementation syntax. The Week 13 review explicitly assesses communication appropriate to the reader. [L13 pp.13-18]

14. Exam preparation and answer checks

14.1 What the supplied review says

L13 pp.5-12 describes an in-person handwritten exam with two hours of writing plus ten minutes of reading, worth 50% of the unit. It reports restricted open book with unlimited handwritten or printed double-sided note sheets, a bilingual dictionary and a non-programmable calculator; no books. It also describes an exam hurdle of at least 40% and an overall mark of at least 50. **These are the statements in the supplied review, not an independent confirmation of your personal timetable, permitted materials or final university rules.** The review itself says official university announcements have authority, and some slide footers retain 2025 text.

Review structure, L13 p.12	Marks	Main preparation
Q1 Conceptual EER design	8	Entities, attributes, relationships and unit-specific notation
Q2 SQL	6	Correct joins, filters, grain, aggregation and ties
Q3 EER to relational schema	8	Complete mapping with keys, FKs and constraints
Q4 Foundations, algebra and internals	6	Precise terminology and access-path reasoning
Q5-Q6 Design theory	4 + 8	Closures, minimal keys, BCNF and decomposition proofs
Q7-Q8 Security and privacy	5 + 5	Policy, views, privileges and application security

Everything taught in lectures, labs and assessments is potentially examinable unless explicitly excluded. The announced question balance is a way to allocate practice, not a reason to discard transactions or analytics. Weekly quizzes revisit earlier topics and are not a complete representation of the written exam. [L13 pp.9-12]

14.2 A practical study sequence

Session 1: represent facts. Draw three models from English descriptions, checking both sides of each relationship. Map them to schemas and explain what the declared constraints cannot enforce. Reconstruct the weak-entity and ISA mappings from memory.

Session 2: derive answers. Write SQL for joins, all-parent counts, distinct counts, ties, correlated subqueries and "every." Translate a subset into relational algebra. Test each answer with a small counterexample, not just the lecture's familiar dataset.

Session 3: prove design properties. Calculate closures until stable. Prove at least two keys including minimality. Give one BCNF violation, carry out a split and justify losslessness. Evaluate dependency preservation separately.

Session 4: explain systems. Trace one Flask request into SQL and back to a template. Explain an injection repair, a view-based policy, an index tradeoff and one transaction anomaly in a focused paragraph each. Review OLAP and governance vocabulary.

Session 5: timed practice. Use the separate eight-question practice exam. Aim for about two minutes per mark on the first pass, leaving time to check and repair answers. The 50-mark practice paper has a 100-minute first pass plus a 20-minute review allowance within the writing time. It is independently written and is not a prediction of the actual exam.

14.3 Before handing in each answer

Answer type	Essential checks
EER	Every required fact is represented; identifiers distinguish instances; arrows point to diamonds; thickness encodes minimum participation; ISA constraints are explicit
Relational schema	Every attribute and association is mapped; composite keys are complete; FK direction and nullability are correct; limitations are acknowledged
SQL	Correct result grain; all needed tables; correct join predicates; zero cases and NULL; GROUP BY/HAVING; DISTINCT only when justified; ties retained
Algebra	Attributes exist at each step; set operands are compatible; join conditions connect the intended facts; projection follows necessary filtering
Theory	All LHS attributes present before an FD fires; closure reaches a fixed point; minimality proved; BCNF determinants tested; common-attribute losslessness proof
Security	Policy clearly stated; mechanism actually enforces it; no base-table bypass; values bound; dynamic identifiers controlled; relevant account distinguished
Internals	Index type, column order, clustering and selectivity all considered; cost assumptions stated; update overhead acknowledged
Transactions	Whole logical unit identified; commit/rollback boundaries correct; anomaly mechanism explained; isolation guarantee matches the invariant

The course review emphasises correct essential logic for SQL/algebra and the taught diagram notation. Use focused explanations that answer the exact request. If asked to justify, give the reason as well as the result. [L13 pp.14-18]

15. Compact reference and glossary

15.1 Query and theory patterns

Need	Pattern
Keep one candidate if any match exists	EXISTS with a correlation to candidate key
Keep candidates with no match	NOT EXISTS with the intended correlation
Match every required item	No required item exists for which no match exists
Count all parents including zero	LEFT JOIN; child filter in ON; COUNT(child non-NULL key)
Count entities after a multiplying join	COUNT(DISTINCT entity key), or avoid multiplication with EXISTS
Return all ties for maximum	value = (SELECT MAX(value) over the same qualifying population)
Determine if X -> Y is implied	Calculate X+; test that every attribute of Y is included
Determine if X is a candidate key	X+ = R, and no attribute can be removed while retaining that property
Disprove BCNF	Find nontrivial X -> Y with X+ not equal to R
Prove binary losslessness	R1 intersection R2 determines R1 or R2
Check dependency preservation	Union the projected FDs; test implication of each original FD

15.2 Terms that are often confused

Term	Precise reminder
Schema / instance	Design and constraints / current contents
Relation / relationship	Mathematical table-like structure / conceptual association
Selection / projection	Rows / attributes, with duplicate elimination in set algebra
Composite / multivalued attribute	Several components / several values
Superkey / candidate key	Determines all attributes / minimal such determinant
Primary key / search key	Chosen logical identifier / indexed attribute sequence
Lossless / dependency-preserving	Reconstructs facts exactly / preserves local constraint enforcement
Authentication / authorisation	Establish identity / decide permitted actions
Security integrity / semantic integrity	Who may modify / which data states are valid
View / materialised result	Stored definition / stored data requiring its own maintenance
Connection / cursor	DBMS session / execution and row-result interface
Bound value / SQL identifier	Data parameter / part of query structure
Optimisation / tuning	Plan for a query / design for a workload
Clustered / covering	Physical locality / contains required query values
Atomicity / isolation	All-or-none effects / separation of concurrent tasks
OLTP / OLAP	Operational requests / analytical exploration
Measure / dimension	Quantity being analysed / axis used to organise it
Governance / access control	Decision rights and accountability / one enforcement mechanism

15.3 Twelve fast recall prompts

1. Why is a foreign key not an equality of the parent and child value sets? 2. Which end of an ISYS2120 ER edge contains the arrowhead? 3. What extra constraint turns an optional many-to-one FK into mandatory participation? 4. Why does a left join with COUNT(*) fail for zero children? 5. What makes a subquery correlated? 6. What happens to "every" when the requirement set is empty? 7. Can an observed instance prove an FD for all legal states? 8. Why does candidate-key proof need minimality? 9. What determines the losslessness of a binary split? 10. Why does a value placeholder not protect a user-supplied column name? 11. How does (a,b) differ from (b,a)? 12. Why can individually consistent transactions still break a rule under weaker isolation?

16. Source index and verification notes

16.1 Local lecture sources

Code	Local PDF in Lec/	Main coverage in this guide
L1A	01A_Intro-2026.pdf	Course context and historical quiz administration
L1B	01B_Overview-2026.pdf	Foundations, abstraction, roles and DBMS benefits
L2A	02A_ER_Diagrams-2026-v1.1.pdf	Relational identifiers, ER notation, weak entities, ISA, aggregation
L2B	02B_SQL-Basics-2026.pdf	SQL retrieval, joins, strings, aggregation
L3A	03A-SQL-DDL-2026-v1.1.pdf	Types, DDL, keys, FKs, checks and modifications
L3B	03B-ER-to-SQL-2026.pdf	Mapping conceptual models into relations
L4A	04A-RelationalAlgebra-2026.pdf	Relational operators, composition and equivalences
L4B	04B-more_SQL-2026.pdf	Grouping, NULL, outer joins, subqueries and universal queries
L5	05-conceptmodel-2026.pdf	Natural-language requirements and modelling checks
L6	06-LogicalDesignTheory-2026.pdf	FDs, closure, keys, BCNF and decomposition
L7	07-Security-2026.pdf	Privileges, views, roles, constraints and stored logic
L8A	08A-DB_Applications-2026.pdf	Web architecture and database APIs
L8B	08B-DB_Application-security-2026.pdf	Accounts, injection, XSS and information leakage
L9	09-Internals_Indexing_2026.pdf	Pages, buffering, file organisation and indexes
L10	10-query-processing-2026.pdf	Access paths, cost reasoning, optimisation and tuning
L11	11-Transactions-2025.pdf	Transaction code, ACID, recovery and isolation

Code	Local PDF in Lec/	Main coverage in this guide
L12	12-Analytics_Governance-2026.pdf	Warehousing, OLAP, windows and governance
L13	L13_UoS Review-2026.pdf	Reported exam structure and integrated examples

16.2 Tutorials, revision paper and application source

The tutorial files consulted are the local sheets for Weeks 2-8 and 10-13 in Tut/. The separate root-level ISYS2120 revision questions-2026.pdf supplies the activity schema, airport EER diagram, DoctorsInfo dependencies and Flask-review question. Its 18 subquestions are reproduced in the companion bank with independently written solutions; those solutions are not presented as a published official marking key.

The Week 8 source comes from Tut/isys2120_airline_webapp_2025.zip. The functions in Chapter 9 were checked directly in routes.py and database.py. The application was not deployed or attacked. Findings describe visible construction paths; they do not claim a successful exploit or complete security audit.

16.3 Canvas coverage and answer evidence

The authenticated course was ISYS2120 Data and Information Management, course 73755. The Quizzes page listed ten closed quizzes: Week 2, Week 3 Early Feedback, Weeks 5, 6, 7, 8, 10, 11, 12 and 13. Each visible completed submission contained ten single-choice questions. The captured review set contains 100 question occurrences, each with one explicitly marked Canvas answer. The visible selections match the published key in 68 occurrences and differ in 32. This is a description of these reviews, not a prediction of future marks.

No Week 1, 4 or 9 weekly quiz was listed on that Canvas page. No unseen pool variants are claimed. The available reviews showed one attempt per quiz; the companion bank uses that visible submission and its option order. Five original quiz images were saved through the browser and embedded in the bank, including repeated uses. Original spelling and naming inconsistencies are retained in questions and explained separately.

Technical checks identified issues in the ambiguous unqualified SuppID query, inconsistent Catalog/Catalogue and color/colour names, malformed tuple punctuation and the Week 13 superkey single-choice item with two mathematically valid options. Such notes are explanatory corrections, not changes to the recorded Canvas key. Course material is the scope source; no external contemporary legal, exam-policy or product claims have been independently verified.

17. Practical SQL workshop | Ed Lessons

Use this workshop with Chapters 3-5. It converts selected ISYS2120 Ed Lessons and practice challenges into a method you can apply to unfamiliar questions. Work each example by identifying the required output, row grain, relevant keys, filters, grouping and tie rules.

The shared FilmDB model

Film(film_id, title, release_year, rental_rate, replacement_cost, rating, language_id, original_language_id); Actor(actor_id, first_name, last_name, nationality); Film_Actor(film_id, actor_id); Category(category_id, name); Film_Category(film_id, category_id); Language(language_id, name); Country(country_id, short_code, name). The two bridge tables represent many-to-many relationships. Actor.nationality references Country.short_code.

A repeatable six-step method

1. Underline the output columns and decide what one output row represents. 2. Trace the keys needed to connect tables. 3. Select inner, outer, anti- or self-join. 4. Separate row filters from group filters. 5. Specify duplicate, NULL, tie and empty-set behavior. 6. Add final ordering and test a small counterexample.

Ed sequence and scope

The Ed catalogue lists 16 SQL topics: foundations and FilmDB; DML/DDL; filtering, ordering, patterns and NULL; joins and set operations; arithmetic, strings and time; subqueries, grouping, ranking, conditional expressions and advanced SQL. The worked examples that follow are independently written explanations based on selected visible practice challenges; they are not an official solution key.

Open the consolidated Ed SQL practice sequence (course login required). SQL uses PostgreSQL conventions. Exercise links identify the source prompts; do not assume the examples were submitted or marked in Ed.

17.1 | Tables, keys and views

Build a table from the constraints

Translate identity to a primary key, required attributes to NOT NULL, and references to foreign keys. A nullable foreign key permits the relationship to be absent; it does not permit an invalid non-NULL reference.

```
CREATE TABLE Person (
  license VARCHAR(20) PRIMARY KEY,
  name    VARCHAR(50) NOT NULL
);
CREATE TABLE Car (
  regno   CHAR(6) PRIMARY KEY,
  model   VARCHAR(30),
  driver  VARCHAR(20) REFERENCES Person(license)
);
```

Each car has at most one driver because it has one driver value. A person may drive multiple cars; add UNIQUE(driver) only if the requirements also restrict each person to at most one car. The license type is an explicit modelling assumption.

Ed practice: Creating Tables

A reusable projection and filter

```
CREATE VIEW FilmCostList AS
SELECT title, release_year AS year, rental_rate AS costs
FROM Film
WHERE original_language_id IS NULL;
```

A view stores a query definition. Apply ORDER BY when querying the view: `SELECT * FROM FilmCostList ORDER BY title, year;` An outer query without ORDER BY has no guaranteed order, even if a view definition includes one.

Ed practice: Simple Film View

17.2 | Filter, sort and handle missing values

Ranges, Boolean logic and patterns

```
SELECT film_id, title, release_year
FROM Film
WHERE release_year BETWEEN 2002 AND 2006
  AND (title LIKE 'A%' OR title LIKE 'B%')
ORDER BY release_year DESC, title ASC;
```

BETWEEN includes both endpoints. Parentheses define the OR group. LIKE uses % for any sequence and _ for one character; PostgreSQL ILIKE is case-insensitive. DISTINCT removes duplicate projected rows, not duplicate source entities.

Missing is not a regular value

```
SELECT film_id, title, release_year,
       COALESCE(rating, 'TBD') AS rating
FROM Film
ORDER BY release_year DESC, title;
```

Use IS NULL and IS NOT NULL, not = NULL. COALESCE replaces NULL in the output, not in stored rows. Conversely NULLIF(rating, 'TBD') returns NULL when rating is TBD. Comparisons with NULL produce UNKNOWN and are not retained by WHERE.

Ed practice: NULL to TBD

Ed practice: Deja Vu

Arithmetic and text

```
SELECT 355.0 / 113 AS approximation;
SELECT actor_id, last_name || ', ' || first_name AS fullname
FROM Actor WHERE last_name LIKE 'A%';
```

In PostgreSQL integer / integer truncates, so introduce a numeric operand before dividing. Concatenation involving NULL yields NULL; use COALESCE only when replacing missing text is intended.

Ed practice: A Slice of Pi

Ed practice: Concatenated Names

17.3 | Join paths and preserving rows

Films and their actors

```
SELECT f.film_id, f.title, a.actor_id,
       a.first_name, a.last_name
FROM Film AS f
JOIN Film_Actor AS fa ON fa.film_id = f.film_id
JOIN Actor AS a ON a.actor_id = fa.actor_id
WHERE f.release_year = 2004
ORDER BY f.title, a.last_name, a.first_name;
```

The grain is one film-actor association. Joining Film to Actor needs the bridge. DISTINCT is not a substitute for a correct join predicate, and a title is not a reliable film identifier.

Ed practice: Cast of Films from 2004

Count every category, including unused ones

```
SELECT c.category_id, c.name, COUNT(fc.film_id) AS films
FROM Category AS c
LEFT JOIN Film_Category AS fc
  ON fc.category_id = c.category_id
GROUP BY c.category_id, c.name
ORDER BY films DESC, c.name;
```

LEFT JOIN preserves a category with no film; COUNT(fc.film_id) counts only matched rows and returns zero. COUNT(*) would count the NULL-extended row as one. Put conditions on the optional side in ON when you still need unmatched left rows.

Ed practice: Film Category Usage

17.4 | Two categories and duplicate names

A film must have both categories

```
SELECT f.film_id, f.title
FROM Film AS f
JOIN Film_Category AS fc ON fc.film_id = f.film_id
JOIN Category AS c ON c.category_id = fc.category_id
WHERE c.name IN ('Drama', 'Family')
GROUP BY f.film_id, f.title
HAVING COUNT(DISTINCT c.name) = 2
ORDER BY f.title, f.film_id;
```

Each bridge row has one category, so testing c.name = Drama AND c.name = Family on one row cannot work. The grouped count requires both distinct names. If names are not unique, verify that names are the intended identifiers.

Ed practice: Find films that have two categories

Find actors who share a first and last name

```
SELECT DISTINCT a.actor_id, a.first_name, a.last_name
FROM Actor AS a CROSS JOIN Actor AS b
WHERE a.first_name = b.first_name
  AND a.last_name = b.last_name
  AND a.actor_id <> b.actor_id
ORDER BY a.first_name, a.last_name, a.actor_id;
```

The CROSS JOIN creates candidate pairs; the WHERE clause retains pairs with equal names and different identities. DISTINCT is needed here because one actor can match several others. Equality does not match NULL names.

Ed practice: Finding Duplicate Actors

17.5 | Sets and anti-joins

Combine country and language names

```
SELECT name FROM Country
UNION
SELECT name FROM Language
ORDER BY name;
```

UNION removes duplicates across and within inputs; UNION ALL retains them. The projections must have the same number of compatible columns. Use INTERSECT for common rows and EXCEPT for rows in the first result but not the second.

Ed practice: List all Country and Language names together

Films with no actors

```
SELECT f.film_id, f.title
FROM Film AS f
WHERE NOT EXISTS (
  SELECT 1 FROM Film_Actor AS fa
  WHERE fa.film_id = f.film_id
)
ORDER BY f.title, f.film_id;
```

NOT EXISTS tests for the absence of a matching row. It remains safe if a subquery column contains NULL. A LEFT JOIN with WHERE fa.film_id IS NULL is also valid when the tested matched column is non-NULL. NOT IN can behave unexpectedly if its result contains NULL.

Ed practice: Films without Actors

17.6 | Subqueries, minima and leap years

Return all cheapest films in the 2006 subset

```
SELECT film_id, title, rental_rate
FROM Film
WHERE release_year = 2006
  AND rental_rate = (
    SELECT MIN(rental_rate) FROM Film
    WHERE release_year = 2006
  )
ORDER BY title, film_id;
```

The inner query calculates the minimum within the same subset. Equality returns every tied minimum. MIN ignores NULL; if no non-NULL rate exists the result is empty. LIMIT 1 would discard ties.

Ed practice: Cheapest Rentals from 2006

Determine leap years using integer release years

```
SELECT COUNT(*) AS leap_films
FROM Film
WHERE MOD(release_year, 400) = 0
   OR (MOD(release_year, 4) = 0
       AND MOD(release_year, 100) <> 0);
```

A century year must be divisible by 400; 2000 is a leap year, 1900 is not, and 2004 is. The FilmDB release_year is an integer year rather than a date; do not extract a year from it.

Ed practice: Leap Films

Dates and times

PostgreSQL DATE subtraction returns an integer number of days. TIMESTAMP carries date and time; TIME carries time of day. Time-zone-aware values depend on the session configuration. For the Ed late-fee prompt, use its stated GREATEST rule rather than assuming a fee policy.

Ed practice: Swanky Late Fees

17.7 | Grouping and the order of operations

Years with the most films

```
SELECT release_year, COUNT(*) AS number_of_films
FROM Film
GROUP BY release_year
ORDER BY number_of_films DESC, release_year ASC;
```

WHERE filters individual rows before groups form. HAVING filters groups after aggregation. SELECT determines projected values; ORDER BY orders the final result. A NULL release_year forms its own group unless excluded.

Ed practice: Release Year Stats

Films with at least five actors

```
SELECT f.film_id, f.title, COUNT(fa.actor_id) AS actors
FROM Film AS f
JOIN Film_Actor AS fa ON fa.film_id = f.film_id
GROUP BY f.film_id, f.title
HAVING COUNT(fa.actor_id) >= 5
ORDER BY f.title, f.film_id;
```

Group by identity as well as title so different films with the same title stay separate. If bridge duplicates are possible, COUNT(DISTINCT fa.actor_id) counts distinct people; FilmDB uses the pair as its key.

Ed practice: At Least Five Actors

Filtered nationality counts

```
SELECT nationality, COUNT(*) AS actor_count
FROM Actor
GROUP BY nationality
HAVING COUNT(*) >= 2
ORDER BY actor_count DESC, nationality ASC;
```

Ed practice: Filtered Nationality Stats

17.8 | Limits, ties and ranking

Top five nationalities by actor count

```
SELECT c.name AS country, COUNT(*) AS num_actors
FROM Actor AS a
JOIN Country AS c ON c.short_code = a.nationality
GROUP BY c.country_id, c.name
ORDER BY num_actors DESC, c.name ASC
LIMIT 5;
```

This returns up to five country rows, including only countries represented by actors. The join uses the FK shown in the Ed prompt. LIMIT returns a fixed maximum, not every tie at the cutoff; add a stable key to ordering if an exact deterministic choice among equally named countries is needed.

Ed practice: Top Five Actor Nationalities

Rank categories by their number of films

```
WITH counts AS (
  SELECT c.category_id, c.name, COUNT(fc.film_id) AS films
  FROM Category AS c
  LEFT JOIN Film_Category AS fc
    ON fc.category_id = c.category_id
  GROUP BY c.category_id, c.name
), ranked AS (
  SELECT name, films, RANK() OVER (ORDER BY films DESC) AS rank
  FROM counts
)
SELECT rank, name AS category, films
FROM ranked WHERE rank <= 5
ORDER BY rank, category;
```

RANK gives equal counts the same rank and leaves gaps; DENSE_RANK leaves no gaps; ROW_NUMBER assigns a distinct position. Filtering rank <= 5 can return more than five rows. The precise Ed task may specify its own tie convention; verify that convention before submission.

Ed practice: Ranking Film Categories

17.9 | Conditional expressions and date arithmetic

Label ratings without changing stored data

```
SELECT film_id, title, rating,
       CASE
         WHEN rating IS NULL THEN 'Unknown'
         WHEN rating = 'G' THEN 'General audience'
         ELSE 'Other rating'
       END AS rating_label
FROM Film
WHERE release_year BETWEEN 2002 AND 2006
ORDER BY title, film_id;
```

CASE is evaluated in the displayed order; the first true branch wins. The labels above illustrate conditional expressions and are not presented as the exact Ed marking criteria.

Ed practice: Interpreting Film Ratings

Translate an explicit late-fee expression

```
SELECT f.film_id, f.title,
       GREATEST(f.rental_rate *
                (DATE '2016-04-12' - DATE '2016-03-24'),
                f.replacement_cost) AS late_fee
FROM Film AS f
JOIN Film_Actor AS fa ON fa.film_id = f.film_id
JOIN Actor AS a ON a.actor_id = fa.actor_id
WHERE a.first_name = 'JOE' AND a.last_name = 'SWANK';
```

The dates differ by 19 days. Follow the question's stated GREATEST comparison even if a different real-world policy seems plausible. If multiple actors share the name, the query includes each matching actor; add DISTINCT only if the requested grain is one film.

Ed practice: Swanky Late Fees

17.10 | Universal conditions: every and none

Actors in every film whose title begins CHOCOLAT

```
SELECT a.actor_id,
       a.first_name || ' ' || a.last_name AS fullname
FROM Actor AS a
WHERE NOT EXISTS (
  SELECT 1 FROM Film AS f
  WHERE f.title LIKE 'CHOCOLAT%'
    AND NOT EXISTS (
      SELECT 1 FROM Film_Actor AS fa
      WHERE fa.actor_id = a.actor_id
        AND fa.film_id = f.film_id
    )
)
ORDER BY fullname, a.actor_id;
```

Read inside out: there is no qualifying film for which this actor has no association. Extra films do not disqualify an actor. If no film matches the prefix, every actor satisfies the condition by vacuous truth; add EXISTS on the target set only if the question requires a non-empty target.

Ed practice: "Mmmm Chocolat"

Equivalent set-based reasoning

For a given actor, target film IDs EXCEPT that actor's film IDs must be empty. For identical sets, check both directions: A EXCEPT B and B EXCEPT A. Checking only one direction establishes containment, not equality.

17.11 | Identical casts and ordered lists

Return films sharing an identical cast with another film

```
SELECT f.film_id, f.title
FROM Film AS f
WHERE EXISTS (
  SELECT 1 FROM Film AS g
  WHERE g.film_id <> f.film_id
    AND NOT EXISTS (
      (SELECT actor_id FROM Film_Actor WHERE film_id = f.film_id)
      EXCEPT
      (SELECT actor_id FROM Film_Actor WHERE film_id = g.film_id)
    )
    AND NOT EXISTS (
      (SELECT actor_id FROM Film_Actor WHERE film_id = g.film_id)
      EXCEPT
      (SELECT actor_id FROM Film_Actor WHERE film_id = f.film_id)
    )
)
ORDER BY f.title, f.film_id;
```

EXISTS emits each qualifying outer film once even if several other films match. Two films with no cast have identical empty sets under this definition; add an explicit non-empty-cast test if the task requires it.

Ed practice: Identical Casts

List each film's categories in alphabetical order

```
SELECT f.film_id, f.title,
       string_agg(c.name, ',' ORDER BY c.name) AS categories
FROM Film AS f
LEFT JOIN Film_Category AS fc ON fc.film_id = f.film_id
LEFT JOIN Category AS c ON c.category_id = fc.category_id
GROUP BY f.film_id, f.title;
```

PostgreSQL string_agg ignores NULL values and yields NULL for a film with no categories. Use COALESCE only if a specific replacement is requested. Aggregate ordering belongs inside string_agg; an outer ORDER BY sorts film rows instead.

Ed practice: Film Cat_aggories

17.12 | Multi-step aggregation and test cases

Find every film tied for the most categories

```
WITH stats AS (
  SELECT f.film_id, f.title, COUNT(c.category_id) AS n,
         string_agg(c.name, ',' ORDER BY c.name) AS categories
  FROM Film AS f
  LEFT JOIN Film_Category AS fc ON fc.film_id = f.film_id
  LEFT JOIN Category AS c ON c.category_id = fc.category_id
  GROUP BY f.film_id, f.title
)
SELECT film_id, title, n, categories
FROM stats
WHERE n = (SELECT MAX(n) FROM stats)
ORDER BY title, film_id;
```

The common table expression computes one row per film; the outer query keeps all rows equal to the maximum. If every film has no categories, all films tie at zero. With no films, there are no result rows.

Ed practice: Film Countagories

Run these mental tests before you submit

Zero: one category has no films. Does it survive? Tie: two films have the same minimum. Are both returned? Duplicate: two films share a title. Do they remain distinct? NULL: an optional rating is missing. Is it handled intentionally? Every: the target set is empty. Is vacuous truth appropriate? Order: does the final query state all requested sort keys?

Practical study loop

Attempt each linked Ed challenge before reading the corresponding pattern. Draw the join path, write the query, test a tiny counterexample, then compare with the Ed feedback. Adapt column aliases and exact ordering to each prompt. These illustrative queries were not submitted to Ed and are not a substitute for its marking feedback.